

SQL Set Operations: UNION, INTERSECT, and MINUS (EXCEPT)

Objective

The purpose of this research is to study SQL set operations and understand how they are used to combine or compare the results of multiple queries. The report covers the three main set operations: UNION, INTERSECT, and EXCEPT (MINUS in Oracle), including their definitions, syntax, rules, differences, and practical examples.

1. UNION

Definition

The UNION operator combines the results of two or more SELECT statements into a single result set and removes duplicate rows automatically.

When to Use

UNION is used when data from multiple queries needs to be combined while ensuring that duplicate records appear only once in the final result.

Syntax

```
SELECT column1, column2, ...  
FROM table1  
UNION  
SELECT column1, column2, ...  
FROM table2;
```

Rules and Conditions

1. Both SELECT statements must return the same number of columns.
2. Corresponding columns must have compatible data types.
3. Column names in the final result are taken from the first SELECT statement.
4. Duplicate rows are automatically removed.

Practical Example

```
SELECT employee_id
FROM employees
WHERE department_id = 10
UNION
SELECT employee_id
FROM employees
WHERE department_id = 20;
```

This query returns all employee IDs from departments 10 and 20 without duplicates.

2. INTERSECT

Definition

The INTERSECT operator returns only the rows that exist in both result sets.

When to Use

INTERSECT is useful when identifying common records between two queries.

Syntax

```
SELECT column1, column2, ...
FROM table1
INTERSECT
SELECT column1, column2, ...
FROM table2;
```

Rules and Conditions

1. Both queries must return the same number of columns.
2. Corresponding columns must have compatible data types.
3. Only rows appearing in both result sets are returned.
4. Duplicate rows are removed.

Practical Example

```
SELECT employee_id
FROM employees
WHERE department_id = 10
INTERSECT
SELECT employee_id
FROM employees
WHERE salary > 5000;
```

This query returns employees who belong to department 10 and also earn more than 5000.

3. EXCEPT / MINUS

Definition

The EXCEPT operator returns rows from the first query that do not exist in the second query.

In Oracle Database, the equivalent operator is MINUS.

When to Use

MINUS is used when finding records that exist in one result set but not in another.

Syntax

Standard SQL

```
SELECT column1, column2, ...  
FROM table1  
EXCEPT  
SELECT column1, column2, ...  
FROM table2;
```

Oracle SQL

```
SELECT column1, column2, ...  
FROM table1  
MINUS  
SELECT column1, column2, ...  
FROM table2;
```

Rules and Conditions

1. Both queries must return the same number of columns.
2. Corresponding columns must have compatible data types.
3. Duplicate rows are removed automatically.
4. Oracle uses MINUS instead of EXCEPT.

Practical Example

```
SELECT employee_id  
FROM employees  
WHERE department_id = 10  
MINUS  
SELECT employee_id  
FROM employees  
WHERE salary > 5000;
```

This query returns employees in department 10 whose salary is not greater than 5000.

Difference Between UNION and UNION ALL

Feature	UNION	UNION ALL
Removes Duplicates	Yes	No
Performance	Slower	Faster
Returns All Rows	No	Yes
Duplicate Records	Removed	Retained

Example

UNION

```
SELECT department_id
FROM employees
UNION
SELECT department_id
FROM departments;
```

Duplicate department IDs are removed.

UNION ALL

```
SELECT department_id
FROM employees
UNION ALL
SELECT department_id
FROM departments;
```

Duplicate department IDs are retained.

Demonstration Using Sample Tables

Create Tables

```
CREATE TABLE TABLE_A (
    ID NUMBER,
    NAME VARCHAR2(20)
);

CREATE TABLE TABLE_B (
    ID NUMBER,
    NAME VARCHAR2(20)
);
```

Insert Data

```
INSERT INTO TABLE_A VALUES (1, 'Ali');  
INSERT INTO TABLE_A VALUES (2, 'Sara');  
INSERT INTO TABLE_A VALUES (3, 'Omar');
```

```
INSERT INTO TABLE_B VALUES (2, 'Sara');  
INSERT INTO TABLE_B VALUES (3, 'Omar');  
INSERT INTO TABLE_B VALUES (4, 'Ahmed');
```

UNION Example

```
SELECT * FROM TABLE_A  
UNION  
SELECT * FROM TABLE_B;
```

Result:

	ID	NAME
1	1	Ali
2	2	Sara
3	3	Omar
4	4	Ahmed

INTERSECT Example

```
SELECT * FROM TABLE_A  
INTERSECT  
SELECT * FROM TABLE_B;
```

Result:

	ID	NAME
1	2	Sara
2	3	Omar

MINUS Example

```
SELECT * FROM TABLE_A  
MINUS  
SELECT * FROM TABLE_B;
```

Result:

	ID	NAME
1	1	Ali

Conclusion

SQL set operations provide a powerful way to combine and compare query results. UNION merges result sets while removing duplicates, INTERSECT returns common records, and MINUS (EXCEPT) returns records that exist in the first query but not in the second. Understanding these operations is essential for data analysis, reporting, and advanced SQL querying in Oracle databases.